LuciOS — Technical Implementation Plan

Revision: 1.1 · **Prepared for:** Luci Crow · **Date:** October 22, 2026
Release: Core academic/professional portfolio MVP · **Implementation status:** Not yet started

> **Read first:** `AGENT_HANDOFF.md`. This plan supersedes the radial-dial, free-floating multi-window, and dummy app approaches in older plans. **Do not implement** puzzle systems, the garden, page peel or evilLuciOS as part of this build.

1. Goal, constraints and priorities

Build and publish an accessible React-based portfolio that feels like a wholly original steampunk-cyberpunk computer. It presents honest technical evidence using a mechanical app delivery metaphor: **hanging overhead conveyor tags → robotic retrieval → piped black display frame → holographic app**.

Primary deadline constraint: The owner is planning around roughly three working days plus a potential two-day grace period. Prioritize **deployed, rubric-complete content** before the most elaborate visual flourish. The grace period is contingency, not planned feature expansion.

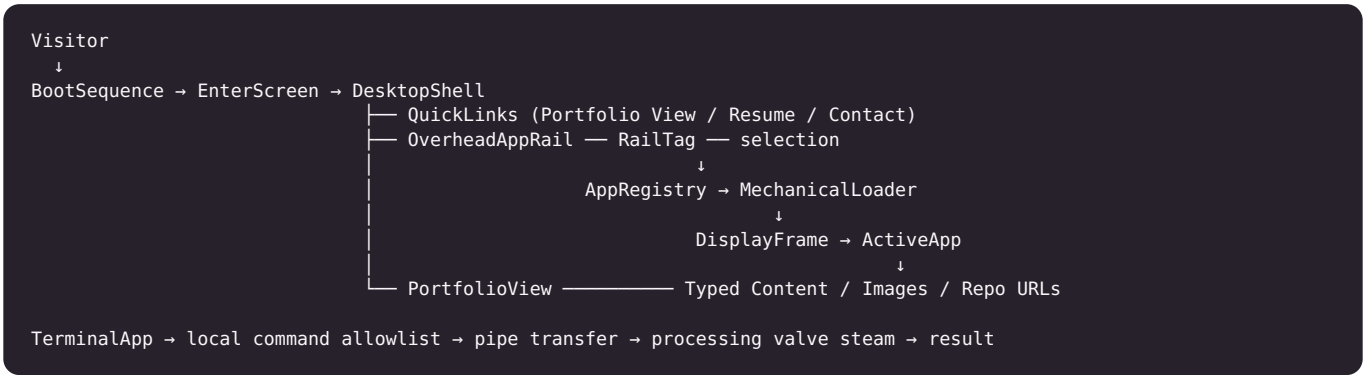
Order of priority:

1. **P0:** Accurate About, Projects, Skills, Resume, Contact, Portfolio View, real project images/code and repository links; builds and deploys.
2. **P1:** Overhead mechanical app rail and single framed app display; functional keyboard/pointer interactions; basic loading/closing states.
3. **P2:** Hand pickup, swing physics, gear motion, subtle wallpaper, animated hologram/bronze piping, terminal valve steam.
4. **Future / NOT v1.1:** evilLuciOS, hidden garden/puzzles, page peel, elaborate multi-window manager, fake demo apps, phone-style OS.

2. Stack and dependencies

Area	Proposed technology	Notes
Framework/build	React + TypeScript + Vite	Static client; add only necessary dependencies
UI styles	CSS variables, CSS modules/global styles, SVG	Reusable design tokens; no component library requirement
State	React Context + `useReducer`	Explicit loader transitions and app selection; replace only with justification
Motion	CSS transitions / Web Animations	Cancel/skip on reduced motion or interruption
Data	Typed local data records	Project cases and highlights separate from UI structure
Assets	Optimized genuine PNG/JPEG/WebP screenshots, safe code text/images	No mock project demos
Tests	Vitest + React Testing Library proposed; optional Playwright smoke checks	Prefer targeted acceptance gates
Hosting	HTTPS static service; exact host TBD	Relative asset URLs/SPA fallback reviewed for host
Backend	**None for v1.1**	Contact link instead of server form to save time

3. Architecture, components and dependencies



Component contracts

Module	Owns	Must never do
`BootSequence` / `EnterScreen`	Progress look, accessible Entry/Skip	Imply account/password authentication; force lengthy wait
`DesktopShell`	Wallpaper, app rail layout, direct shortcuts	Hardcode project details
`OverheadAppRail` / `RailTag`	Ordered selection, offset, hover/focus edge scroll and button fallbacks	Hide apps out of keyboard reach
`MechanicalLoader`	`selected → retrieving → lowering → inserting → revealing` animation	Delay or block content on failed animation
`DisplayFrame`	Single active app, bronze frame, top-right pipe, close/back focus restoration	Open many draggable windows or depend on browser popups
`AppRegistry`	Stable app IDs, titles, icon/asset refs, render map	Duplicate project data
`PortfolioView`	Linear accessible content for professional reading	Depend on mechanical rail
`ProjectCaseStudy`	Purpose, role, tools, outcome, gallery, code highlights, repo at bottom	Render a dummy app instead of real evidence
`TerminalApp` / `PipeValveSequence`	Allowlisted local commands and processing animation	Execute remote/local shell, steam while typing

TypeScript contracts (initial)

```
type AppId = 'about' | 'projects' | 'skills' | 'resume' | 'contact' | 'terminal';
type EntryPhase = 'boot' | 'enter' | 'desktop';
type LoaderPhase =
  | 'idle' | 'selected' | 'retrieving' | 'lowering'
  | 'inserting' | 'revealing' | 'open' | 'closing';

type UIState = {
  entryPhase: EntryPhase;
  selectedApp: AppId | null;
  displayOpen: boolean;
  loaderPhase: LoaderPhase;
  railIndex: number;
  reducedMotion: boolean;
};

type UIAction =
  | { type: 'ENTER_DESKTOP' }
  | { type: 'SELECT_APP'; appId: AppId }
  | { type: 'ADVANCE_LOADER'; phase: LoaderPhase }
  | { type: 'OPEN_IMMEDIATELY'; appId: AppId }
  | { type: 'CLOSE_DISPLAY' }
  | { type: 'SET_RAIL_INDEX'; index: number }
  | { type: 'SET_REDUCED_MOTION'; value: boolean };
```

Invariants: one active framed app; selecting another cancels the previous animation; close/back always works; reduced motion can jump straight to `open`; no transition can leave a blank overlay indefinitely. Ensure quick links work regardless of the loader state.

Project data contract

```
type Project = {
  id: 'battleship' | 'corvusdesk' | 'bad-apple';
  title: string;
  purpose: string;
  contributions: string[];
  technologies: string[];
  decisions: string[];
  outcomeOrLessons: string;
  screenshots: { src: string; alt: string; caption: string }[]; // exactly three authentic approved images
  codeHighlights: {
    title: string; language: string;
    code?: string; imageSrc?: string; explanation: string;
  }[];
  repoUrl: string; // public, per-project, verified
};
```

Use a single source of portfolio content for both the framed apps and linear Portfolio View. Do not invent project accomplishments, screenshots, code, or URLs. Development placeholders must be clearly marked and removed before public submission.

4. Proposed repository layout

```
lucios/
├── public/
│   ├── assets/icons/              # original pixel hanging-app icons
│   ├── assets/projects/           # vetted real screenshots + selected code screenshots
│   └── resume/                  # current, public-safe resume PDF
├── src/
│   ├── app/                     # boot/entry, shell, registry
│   ├── components/              # AppRail, MechanicalLoader, DisplayFrame, media viewer
│   ├── state/                   # UI reducer; loader phase guards
│   ├── apps/                    # About, Projects, Skills, Resume, Contact, Terminal
│   ├── content/                 # typed profile, skills, projects, links
│   ├── styles/                  # tokens, layout and motion styles
│   ├── utils/                   # terminal allowlist, focus helpers, reduced-motion detection
│   └── main.tsx
├── tests/
├── docs/                      # planning PDFs, Markdown docs, visual drawups
├── README.md
└── package.json
```

No `demos/` directory: there are no recreated Battleship/CorvusDesk/Bad Apple applications in v1.1.

5. Design and interactions

Core colors: `#14131A` black, `#23212C` charcoal, `#D69DBB` pastel pink, `#F8F7F9` mist white, `#A1A4AB` grey steel, `#CB3553` red Enter. **Bronze piping** is a material/accent role (proposed `#A67549`, adjustable after prototype). Use pastel pink glow for the futuristic panels and brass/bronze mechanical parts for steampunk detail. Retain legible body text.

App rail behavior

- Overhead horizontal track across top; small app tags on hanging hooks using pixel-art faces and readable labels.
- Left/right **edge hover** optionally rotates or scrolls rail; explicit visible arrow buttons also provide click and keyboard actions; focus scrolls chosen tag into view.
- `Enter`/click selects the focused tag; a subtle swing communicates the selection.
- If CSS/JS motion fails, the rail remains a working accessible list of app buttons.

Loader + display behavior

- Stage A select/swing; Stage B articulated hand (simple 2D arm is acceptable) removes tag; Stage C dark display lowers with bronze pipe border (pipe exits upper right/offscreen); Stage D tag inserted, gears and brief processing feedback; Stage E holographic content visible.
- **Minimum shippable effect:** No 3D hand required. A frame-drop transition, decorative rail, and one short gear/steam effect communicate the concept. The real app must open without waiting for a lengthy sequence.
- The app frame uses **one active display**, not draggable/floating OS windows. `Back`/`Close` is clearly labeled, keyboard operable, and visually integrated into the machinery.
- Gears and/or steam may activate on **subsequent meaningful app-loading or loading-state transitions**, never every keystroke and never in a way that delays reading.

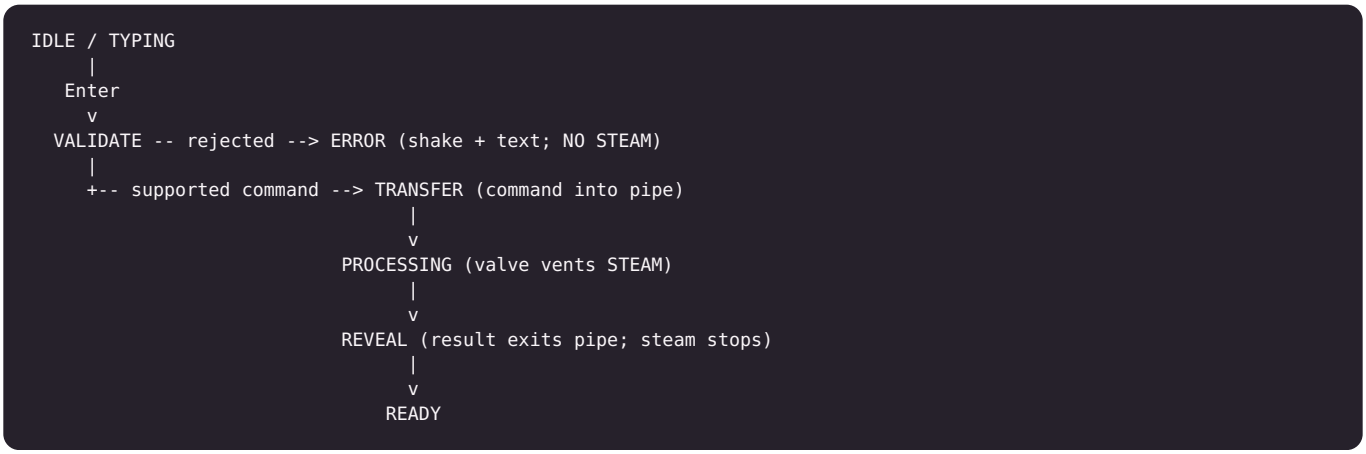
Projects pages

For each of Battleship, CorvusDesk and Bad Apple Repair:

1. Title, purpose, Luci's specific role, stack and technical reasoning.
2. **Screenshots**: three genuine application screenshots in a gallery; click/keyboard to enlarge; captions/alt text.
3. **Code highlights**: selected real excerpts or code images plus a sentence or two about *why* the code matters. Ensure readable source text where possible; no private material.
4. Outcome/lesson.
5. **GitHub repository link at the bottom of this project**, not only on the general GitHub profile page. Check destination and permissions.

Do not construct local interactive demos or substitute mock screenshots as evidence. If project materials are missing, report the evidence gap; do not silently satisfy it with fabrication.

Terminal state machine



- Supported commands: `help`, `about`, `projects`, `skills`, `resume`, `contact`; mapping is local and deterministic.
- **No steam while typing; no steam on invalid command.** Steam is strictly the processing-state indicator after a valid submission.
- Once text output is ready, processing ends. Reduced-motion mode skips travel, steam and shake and still shows the same result/error immediately.
- Never expose shell commands, arbitrary HTML execution, device file access or server requests based on visitor text.

6. Milestones for implementation agents

Phase	Tasks	Acceptance gate
M0 · Baseline	Scaffold React/Vite/TS, design tokens, typed registry/content skeleton, lint/build, copy docs	`npm run build` succeeds; file map reviewed; no invented links
M1 · Rubric-complete content	About, Skills, Resume, Contact, Portfolio View and basic text project case studies	All sections reachable via ordinary navigation; resume/contact functional
M2 · Desktop and hanging rail	Boot/Enter, wallpaper, app rail, visible side scroll buttons/keyboard, route by app ID	Each rail tag opens the correct app without animations
M3 · Mechanical frame	Single display, bronze pipe visual, open/close, progressive swing/hand/frame/hologram	App opens after select; closes safely; reduced-motion fallback passes
M4 · Authentic project evidence	Three screenshots/project, code highlights, explanations, validated repo link underneath	Three full entries; no mock apps; all evidence authorized
M5 · Terminal	Local allowlist, error shake, pipe transfer, valve processing steam, output	Exact timing verified; no unsafe command handling
M6 · QA + release	Link/image audit, keyboard, reduce motion, narrow readable fallback, build/deploy, reflection	Live HTTPS/resume works; instructor rubric verified

Suggested three-day grouping: Day 1 = M0, M1 and initial M2; Day 2 = complete M2/M3/M4; Day 3 = M5 as time allows, M6 and submission. If behind, ship M1/M2 with reduced animation and verified evidence; reserve any grace period for bugs, **not** new functionality.

7. Acceptance test matrix

ID	Test	Pass condition
T-01	Boot/Entry	Skip/Enter works; no password; no endless simulated loading
T-02	Rail	All six tags reachable by pointer and keyboard; edge controls scroll correctly
T-03	Open app	Tag launches its matching content in **one** display frame
T-04	Close/switch	Closing and selecting another app never traps focus or strands overlay
T-05	Reduced motion	No hand, shake, steam or ambient motion; same content accessible
T-06	Resume/contact	From initial screen/Portfolio View, verified actions available without exploring UI
T-07	Project gallery	Three projects each have **three genuine screenshots** with captions/alt, code highlights
T-08	Repo links	A tested **direct GitHub repository URL appears below each project's content**
T-09	Honest evidence	No dummy app, invented achievement, customer data or unapproved screenshot
T-10	Terminal success	Command transfers; valve steams **only processing**; result appears
T-11	Terminal typo	Invalid command shakes + text; no valve/ steam
T-12	Accessibility	Keyboard and plain Portfolio View; readable narrow-screen fallback
T-13	Release	HTTPS URL, resume download, links, lint/build pass; reflection ready

8. Codex implementation protocol

For each task, Codex should identify the **milestone and FR/NFR IDs**, make the smallest coherent diff, verify build and relevant tests, report actual results and describe anything blocked by missing assets. Do not import earlier prototype assumptions (radial dial, multi-window manager, dummy applications), add APIs, or claim unfinished features work.

First Codex task prompt:

> Read `docs/AGENT_HANDOFF.md` and all current v1.1 specifications. Implement **M0 only**: scaffold React/TypeScript/Vite; create a minimal app registry, initial design tokens, typed content placeholders and setup/test scripts. The chosen UI metaphor is a **top rotating hanging app rail feeding a single bronze-piped holographic display frame**. Project pages will show **real screenshots, real code highlights and direct per-project repo links** — no demo replicas. Do not implement effects or later milestones yet. Show changed files, commands run, test/build results and outstanding missing evidence.

9. Open decisions / required owner assets

- Verified public repo URL for **each** project; no URLs should be guessed.
- Three approved screenshots for Battleship, CorvusDesk and Bad Apple Repair **each**; captions/alt text.
- Selected code highlights for each with notes on original contribution and any publication permissions.

- Final professional bio, current resume PDF, public professional contact method and optional LinkedIn.
- Host, custom domain/base path, and whether the project frame uses fixed or elastic dimensions.
- Exact rail spacing, animation duration and degree of physical realism after first prototype.

10. Definition of done

The portfolio is publicly accessible over HTTPS, meets all instructor rubric elements, has honest descriptions and media for three projects, provides three working per-project GitHub links, a downloadable resume, accessible navigation, safe terminal behavior (if shipped), and accurate documentation of its actual features. Presentation effects may be simplified without compromising professional evidence.

Source hierarchy: `AGENT_HANDOFF.md` → v1.1 SRS (requirements) → v1.1 UI/UX spec (appearance and stories) → this Plan (build order) → Project Proposal (long-term vision). Older drafts are superseded. All features described remain plans until code exists.